

Theory note S0 — Why the BER of BPSK in AWGN is $Q\left(\sqrt{2E_b/N_0}\right)$

Chang Young Ham

September 2026 (accompanies `src/s0_bpsk_awgn_ber.py`)

Purpose. S0 is the calibration stage of the repository: the simulator must reproduce a result whose exact answer is known before it is used on problems (S1, S2) that have no closed form. This note derives that answer, maps every symbol to a line of the code, and explains what the “ratio 0.96–1.03” verification actually proves. Mathematics used: Gaussian random variables, conditional probability, the binomial estimator.

1 Signal model

One bit $b \in \{0, 1\}$ is transmitted per symbol. Binary phase-shift keying maps it to

$$s = \begin{cases} +\sqrt{E_b} & b = 1 \\ -\sqrt{E_b} & b = 0 \end{cases}$$

so that every symbol carries energy $s^2 = E_b$ (energy per bit). The channel adds white Gaussian noise. After the receiver’s matched filter the decision variable is a single real number

$$r = s + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Where $\sigma^2 = N_0/2$ comes from. N_0 is the *one-sided* noise power spectral density (power per Hz of positive frequency). A real matched filter for a unit-energy pulse has a noise output variance equal to $N_0/2$ — half of the density, because the two-sided spectrum spreads N_0 over positive and negative frequencies at $N_0/2$ each. Hence

$$\sigma^2 = \frac{N_0}{2}.$$

This factor of two is the single most common source of a simulated BER curve that is “a few dB off” from theory.

2 The decision rule and the error event

Both symbols are equally likely, and the noise is symmetric, so the maximum-likelihood decision is a threshold at zero:

$$\hat{b} = \begin{cases} 1 & r > 0 \\ 0 & r \leq 0. \end{cases}$$

Condition on $b = 1$ (so $s = +\sqrt{E_b}$). An error occurs when $r \leq 0$, i.e. when $n \leq -\sqrt{E_b}$:

$$P(\text{error} \mid b = 1) = P\left(n \leq -\sqrt{E_b}\right) = P\left(\frac{n}{\sigma} \leq -\frac{\sqrt{E_b}}{\sigma}\right) = Q\left(\frac{\sqrt{E_b}}{\sigma}\right),$$

where $Q(x) = P(Z > x)$ for $Z \sim \mathcal{N}(0, 1)$ is the Gaussian tail function and we used the symmetry $P(Z \leq -x) = P(Z \geq x)$. By the same argument $P(\text{error} \mid b = 0)$ is identical, so the average bit error rate is

$$\text{BER} = Q\left(\frac{\sqrt{E_b}}{\sigma}\right) = Q\left(\sqrt{\frac{E_b}{N_0/2}}\right) = \boxed{Q\left(\sqrt{\frac{2E_b}{N_0}}\right)}. \quad (1)$$

Stop here and check. Put $E_b = 1$, $E_b/N_0 = 10$ (10 dB). Then $\sigma = \sqrt{1/20} = 0.2236$ and the argument of Q is $1/0.2236 = 4.47$. Confirm from a table or code that $Q(4.47) \approx 3.9 \times 10^{-6}$, which is the last point of the S0 curve.

3 Q and erfc

The code uses `scipy.special.erfc` because SciPy does not ship Q directly. The two are related by

$$Q(x) = \frac{1}{2} \operatorname{erfc}\left(\frac{x}{\sqrt{2}}\right) \implies \text{BER} = \frac{1}{2} \operatorname{erfc}\left(\sqrt{\frac{E_b}{N_0}}\right),$$

which is exactly `ber_bpsk_theory()` in the script.

4 Correspondence with the code

The script fixes $E_b = 1$, so $N_0 = 1/(E_b/N_0)$ and $\sigma = \sqrt{N_0/2}$.

Mathematics	Line in <code>s0_bpsk_awgn_ber.py</code>
E_b/N_0 (linear) from dB	<code>ebn0_lin = db_to_linear(ebn0_db)</code>
$\sigma = \sqrt{1/(2 E_b/N_0)}$	<code>sigma = np.sqrt(1.0 / (2.0 * ebn0_lin))</code>
$b \in \{0, 1\}$	<code>bits = rng.integers(0, 2, size=...)</code>
$s = \pm 1$	<code>symbols = 2 * bits - 1</code>
$n \sim \mathcal{N}(0, \sigma^2)$	<code>noise = rng.normal(0.0, scale=sigma, ...)</code>
$r = s + n$	<code>received = symbols + noise</code>
$\hat{b} = [r > 0]$	<code>decisions = (received > 0).astype(int)</code>
count $\hat{b} \neq b$	<code>n_errors += np.sum(decisions != bits)</code>
Eq. (1)	<code>0.5 * erfc(np.sqrt(ebn0_linear))</code>

Note that `scale` in `rng.normal` is the standard deviation σ , not the variance σ^2 .

5 What the Monte Carlo estimate is, and how accurate

The simulator observes K errors in n bits and reports $\widehat{\text{BER}} = K/n$. Each bit is an independent Bernoulli trial with success probability $p = \text{BER}$, so $K \sim \text{Binomial}(n, p)$ and

$$\mathbb{E}[\widehat{\text{BER}}] = p, \quad \text{Var}[\widehat{\text{BER}}] = \frac{p(1-p)}{n} \approx \frac{p}{n}.$$

The *relative* standard deviation is therefore

$$\frac{\sqrt{p/n}}{p} = \frac{1}{\sqrt{np}} = \frac{1}{\sqrt{\mathbb{E}[K]}}.$$

This is why the script stops each point after `MIN_ERRORS = 200` errors rather than after a fixed number of bits: 200 observed errors give a relative spread of $1/\sqrt{200} \approx 7\%$ regardless of how small p is. It also explains why the low-BER points need far more bits (six million at 9 dB versus one million at 0 dB): the required $n \approx 200/p$ grows as p shrinks.

Why the pass band is $0.7 \leq \text{ratio} \leq 1.4$. With a 7 % relative spread, a ratio outside $[0.7, 1.4]$ is roughly a four-standard-deviation event — essentially impossible by chance across ten points. A ratio outside that band, or all ratios biased to one side, therefore indicates a modelling error (usually σ), not statistical noise. The observed range 0.957–1.025 is what an unbiased estimator with $\sim 7\%$ spread looks like.

6 Why the curve is nearly a straight line on a log axis

For large x the Gaussian tail satisfies

$$Q(x) \approx \frac{1}{x\sqrt{2\pi}} e^{-x^2/2},$$

so with $x^2 = 2E_b/N_0$ the BER falls like e^{-E_b/N_0} : exponentially in the *linear* E_b/N_0 . On a log- y axis against E_b/N_0 in dB this is not exactly straight, but close to it over the plotted range, and each additional dB buys roughly a factor of 1.26 in E_b/N_0 and a much larger factor in BER at the high-SNR end. This exponential decay is the benchmark that partial-band jamming destroys in S1, where the decay becomes only inverse-linear.

7 Self-check (answer without this note)

1. Why is the noise variance $N_0/2$ and not N_0 ? What happens to the simulated curve if the code uses $\sigma^2 = N_0$?
2. Derive $P(\text{error} | b = 1)$ in three lines.
3. If you observe 20 errors instead of 200, what is the relative spread of the estimate, and how would the plotted points look?
4. At $E_b/N_0 = 9$ dB the script used 6 million bits. Explain the number.
5. Why is $Q(\sqrt{2E_b/N_0})$ the same as $\frac{1}{2} \text{erfc}(\sqrt{E_b/N_0})$?